

## 3.17. La carte « joker »

3.17 : la carte « joker » — quatre couleurs, une étoile. Dessiné une fois en  $16 \times 16$ , montré à sa taille puis trois fois plus grand avec `sprite16(..., 3)`.

### 1. Ce qu'il faut comprendre

**La carte « joker » : quatre couleurs, une étoile.** Une image de  $16 \times 16$ , dessinée **une seule fois**, dans `perso_JOKER.cpp`.

**Le programme est celui du 3.01** : seuls changent le fichier du dessin et le nom. `sprite16(4, 88, 36, JOKER, 3)` la montre trois fois plus grande.

**Essaie** : change le `3` en `2` ; ou modifie le dessin, et regarde les deux tailles changer ensemble.

#### Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction `texte\(\)` — écrire un mot](#)
- `#include <sprite16>` → [35.2. La fonction `sprite16\(\)` — un personnage de  \$16 \times 16\$](#)
- `#include <Perso>` → [35.1. Perso — dessiner des personnages](#)

### 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1765 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

|                    |       |                                   |                                                     |
|--------------------|-------|-----------------------------------|-----------------------------------------------------|
| démarrage          | 57 o  | ← la console, à l'allumage        | — la mise en route de la console                    |
| main()             | 807 o | ← démarrage                       | — ta fonction                                       |
| VBlank             | 8 o   | ← la console, à chaque image      | — compte les images, 60 fois par seconde            |
| AttendreImage      | 28 o  | ← <code>image()</code>            | — attend l'image suivante                           |
| AttendreVBlank     | 16 o  | ← démarrage                       | — attend que l'écran soit libre                     |
| AttendreAcces      | 12 o  | ← <code>EcrireTexte</code>        | — attend de pouvoir écrire une case                 |
| EcrireTexte        | 10 o  | ← <code>texte()</code>            | — écrit un texte, lettre par lettre                 |
| RangerLutins       | 11 o  | ← démarrage                       | — range les 40 lutins hors de l'écran               |
| InstallerTransfert | 15 o  | ← démarrage                       | — installe la copie des lutins en mémoire rapide    |
| CopierTuiles       | 94 o  | ← démarrage                       | — copie les lettres et les dessins en mémoire vidéo |
| EffacerCarte       | 14 o  | ← démarrage                       | — vide les deux cartes de l'écran                   |
| Tuiles             | 672 o | ← <code>CopierTuiles</code>       | — 8 lettres sur 43, et 34 tuiles dessinées          |
| DonneesTransfert   | 10 o  | ← <code>InstallerTransfert</code> | — la petite routine qui copie les lutins            |
| "CARTE JOKER"      | 11 o  | ← <code>texte()</code>            | — un texte gravé                                    |

Lettres gravées : ACEJKORT

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `Notes`

Fonctions du langage appelées :

|                             |      |                                                                             |
|-----------------------------|------|-----------------------------------------------------------------------------|
| <code>sprite()</code>       | x 34 | 786 o sur place, dans <code>main()</code>                                   |
| <code>texte()</code>        | x 1  | 11 o sur place, dans <code>main()</code> + <code>EcrireTexte</code> (10 o)  |
| <code>image()</code>        | x 1  | 6 o sur place, dans <code>main()</code> + <code>AttendreImage</code> (28 o) |
| <code>sprite16()</code>     | x 2  | 0 o sur place, dans <code>main()</code>                                     |
| <code>spriteTaille()</code> | x 1  | 0 o sur place, dans <code>main()</code>                                     |

## 1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

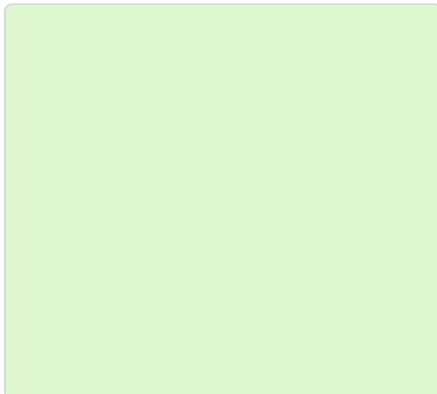
```
// ---- 3.17 La carte « joker » ----  
// Dessiné UNE fois, en 16 × 16. À gauche à sa taille, à droite trois fois plus grand.  
  
#include <texte>           // texte() : écrire un mot  
#include <sprite16>        // sprite16() : poser un dessin de 16 × 16 – et l'agrandir  
#include <Perso>           // Perso : un dessin de 16 × 16  
#include "perso_JOKER.cpp" // le dessin, dans son fichier
```

## 2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

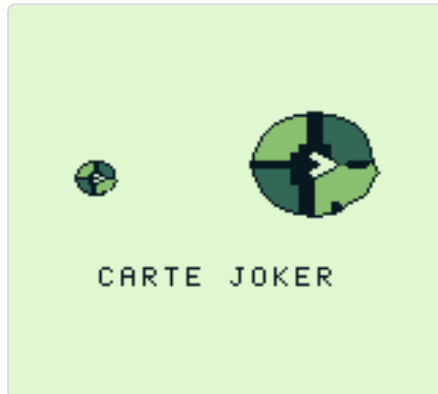
```
int main() {  
    sprite16(0, 24, 56, JOKER);    // taille 1 : les lutins 0 à 3  
    sprite16(4, 88, 36, JOKER, 3); // taille 3 : 48 × 48, à partir du lutin 4  
    texte(4, 12, "CARTE JOKER");  
  
    while (true) {  
        image();  
    }  
}
```

## 3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

## 4. Ce qu'on doit voir

La carte « joker » deux fois : petit à gauche, trois fois plus grand à droite, et CARTE JOKER dessous.

## 5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ taille 1 : l'image en (24, 56)

✓ taille 3 : des lutins dans le carré de 48 à droite (lutin 4 en 96, 36)

✓ le nom est écrit : CARTE JOKER

---

C'est la dernière des 152 leçons. · [gameboy3](#) — écrire une cartouche Game Boy en C++